Software Design Description (SDD)
CODEASTRA 2026 – Online Coding Contest Platform

Team Details :

- Sai Vignesh Budati (2312127) –> Team Leader.
- Ganna Nandan (2312005)
- Rohith Karuturi (2312091)
- Yakkala Bharadwaj (2312104)
- Chimata Teja Sai (2312006)
- K.Venkata Sujith Royal (2312115)
- Lokender Rao Maddila (2312110)
- Muppidi Dhanush (2312160)
- Akkala Varshit Sai (2312156)
- S.Sathwik Reddy (2312151)

1.Introduction

1.1 Purpose
This Software Design Description (SDD) explains the full design of the **CODEASTRA 2026 Online Coding Contest Platform**. It describes the system's modules, workflows, data structures, interfaces, and concurrent processes required to conduct online programming contests with automated code evaluation and real-time leaderboards.

1.2 Scope
The system supports:
- User registration and login
- Contest browsing and participation
- Problem viewing
- Code submission
- Automated judging (compile, execute, evaluate)
- Leaderboard generation
- Admin control for contest and problem management

It focuses on **competitive programming contest management** and **automated evaluation**, not on long-term code storage or discussion forums.

1.3 Definitions
- **SDD** – Software Design Description
- **Participant/User** – Registered user solving coding problems
- **Admin** – User who manages contests, problems, and users
- **Judge Engine** – Automated system for compiling and executing code
- **Contest** – A time-bounded coding event with multiple problems
- **Problem** – Individual coding question with input/output and constraints

- **Submission** – User's uploaded/written code for a problem

---

2. References

1. IEEE 1016 – IEEE Standard for Software Design Descriptions
2. CodeAstra 2026 Hackathon Website Link : https://codeastra2026-web.web.app
3. GitHub Link : https://github.com/Vicky-bud/codeastra-2026

3.1 Module Decomposition

3.1.1 Authentication Module

- Handles user registration and login using email and password.
- Verifies user identity and provides secure access.
- Maintains active user sessions.
- Assigns roles (Participant, Admin).
- Protects restricted areas (Admin panel, judge controls).

3.1.2 Contest Management Module (Admin)

- Allows Admin to create, edit, and delete contests.
- Stores contest metadata (title, description, start & end time, status).
- Controls contest visibility (upcoming, active, completed).
- Provides contest list to participants and other modules.

3.1.3 Problem Management Module (Admin)

- Allows Admin to create and manage problems for contests.
- Stores problem statement, constraints, input/output formats, sample I/O.
- Manages association between problems and contests.
- Manages test cases (input/output pairs) for judge engine.

3.1.4 Dashboard & Contest View Module (Participant)

- Main interface after participant login.
- Displays list of contests (upcoming/active/past).
- Shows contest details and problem lists.
- Provides quick access to submissions and leaderboard.

3.1.5 Problem Display & Code Editor Module

- Displays full problem details to the user.
- Provides code entry area (online editor) or file upload option.
- Allows user to choose programming language.
- Provides "Submit" action to send code to backend.

3.1.6 Submission & Evaluation Module

- Accepts user code submissions.
- Stores submission metadata (user, problem, contest, language, timestamp).
- Sends submission to Judge Engine for compilation and execution.
- Receives result (Accepted, Wrong Answer, TLE, RE, CE, etc.).
- Updates submission status and notifies user.

3.1.7 Judge Engine Module

- Compiles and executes user code in a sandboxed environment.
- Runs code against hidden and sample test cases.

- Enforces time and memory limits.
- Compares program output with expected output.
- Returns evaluation result and execution time to backend.

3.1.8 Leaderboard Module

- Computes rankings for each contest.
- Aggregates scores per user across all problems in that contest.
- Handles tie-breaking (e.g., time-based or penalty-based, as defined).
- Updates ranking in real time after accepted submissions.
- Provides leaderboard view to dashboard and contest pages.

3.1.9 Logging & Monitoring Module

- Records important system events (logins, submissions, errors, judge results).
- Helps in debugging and system auditing.
- Can be extended to support performance monitoring.

3.2 Concurrent Process Decomposition

3.2.1 User Registration & Login Process

- Multiple users can register or log in at the same time.
- The system processes each authentication request independently.
- Prevents duplicate accounts using unique email constraint.
- Maintains responsiveness under concurrent logins.

3.2.2 Contest & Problem Access Process

- Many users may open contest and problem pages simultaneously.
- Contest details and problem statements are fetched in parallel without conflict.
- Caching or read-optimized queries ensure fast responses.

3.2.3 Code Submission Process

- Multiple users submit code at the same time for various problems.
- Each submission is saved as a separate record with its own status.
- Submissions are queued and passed to Judge Engine individually.
- System ensures no submission overwrites another.

3.2.4 Judge Evaluation Process

- Several submissions can be evaluated concurrently.
- Each judge task runs in an isolated execution environment.
- Timeouts prevent infinite loops from blocking the system.
- Results are stored independently to avoid conflicts.

3.2.5 Leaderboard Update Process

- Leaderboard recalculates ranking whenever a relevant submission is evaluated.
- Handles multiple score updates at the same time.
- Resolves ties using defined rules (e.g., score first, then time).
- Ensures participants always see the current and consistent ranks.

3.3 Data Decomposition

3.3.1 User Entity

- Stores user credentials and role information.
- Used for authentication, authorization, and profile display.

Fields:
user_id, name, email, password_hash, role, created_at

3.3.2 Contest Entity

- Stores contest details and schedule.
- Used for contest listing, access control, and leaderboard grouping.

Fields:
contest_id, title, description, start_time, end_time, status, created_at

3.3.3 Problem Entity

- Stores coding problem information.
- Used by problem display, submission module, and judge engine.

Fields:
problem_id, contest_id, title, statement, input_format, output_format, constraints, sample_input, sample_output, max_score, created_at

3.3.4 Submission Entity

- Stores all user submissions for evaluation and history.
- Used by judge engine, result display, and leaderboard calculation.

Fields:
submission_id, user_id, contest_id, problem_id, language, code, result, execution_time, score_obtained, submitted_at

3.3.5 TestCase Entity (if stored in DB; else paths)

- Stores metadata of test cases for each problem.
- Used by judge engine to retrieve corresponding input/output pairs.

Fields:
testcase_id, problem_id, input_path, output_path, is_sample, created_at

3.3.6 Leaderboard Entity

- Stores aggregated results per contest.
- Used for fast retrieval of ranking and scores.

Fields:
leaderboard_id, user_id, contest_id, total_score, penalty_time, last_updated

---

4. Dependency Description

The overall control flow and dependency between main modules is:
Authentication → Dashboard → Contest View → Problem View → Submission & Evaluation → Judge Engine → Leaderboard

- Authentication must be successful before accessing Dashboard.
- Contest Management provides contests to Dashboard and Problem modules.
- Problem Management provides problems to Problem View and Submission modules.
- Submission Module depends on Judge Engine to compute result.
- Leaderboard Module depends on Submissions and Contest/Problem entities.

---

5. Interface Description

5.1 Authentication Interfaces

- registerUser(name, email, password)
  - Creates new user account with encrypted password.
- loginUser(email, password)
  - Authenticates user and starts session.
- logoutUser(user_id)
  - Ends active user session.

5.2 Contest Interfaces

- getContests(status)
  - Returns contests filtered by status (upcoming/active/completed).
- createContest(title, description, start_time, end_time) *(Admin)*
  - Creates a new contest.
- updateContest(contest_id, fields) *(Admin)*
  - Modifies contest details.

5.3 Problem Interfaces

- getProblemsByContest(contest_id)
  - Returns all problems belonging to a contest.
- getProblemDetails(problem_id)
  - Returns complete problem statement and metadata.
- createProblem(contest_id, title, statement, ...) *(Admin)*
  - Adds a new problem.

5.4 Submission Interfaces

- submitCode(user_id, contest_id, problem_id, language, code)
  - Creates a submission and triggers evaluation.
- getSubmission(submission_id)
  - Returns status and result of a specific submission.
- getUserSubmissions(user_id, contest_id)
  - Returns all submissions by a user in a contest.

5.5 Judge Engine Interfaces

- evaluateSubmission(submission_id)
  - Retrieves submission, runs code, stores result.
- Internal (Judge Engine side):
  - compileAndRun(code, language, testcases, limits)
  - Returns {status, execution_time, score_obtained}.

5.6 Leaderboard Interfaces

- getLeaderboard(contest_id)
  - Returns sorted ranking for the contest.
- updateLeaderboard(submission_id)
  - Updates leaderboard based on newly evaluated submission.

6. Detailed Design

6.1 Authentication Module

- Securely hashes passwords before storing.
- Validates email/password combination during login.
- Uses session or token to track authenticated users.
- Restricts access to admin/admin routes based on role.

6.2 Contest Management Module

- Admin creates contests with start and end time.
- System prevents editing contest beyond certain states (e.g., after completion).
- Status transitions: Draft → Upcoming → Active → Completed.
- Provides contest list to dashboard and problem module.

6.3 Problem Management Module

- Admin defines problem metadata and attaches test cases.
- Enforces that each problem belongs to a valid contest.
- Prevents deletion if submissions already exist (or handles cascade rules).

6.4 Dashboard & Contest View Module

- After login, loads all contests relevant to user.
- Segregates contests into sections (Upcoming/Active/Completed).
- Shows quick stats (e.g., number of problems solved).

6.5 Problem Display & Code Editor Module

- Fetches problem data via getProblemDetails(problem_id).
- Shows statement, sample I/O, constraints clearly.
- Provides text area/editor and language dropdown.
- On submit, calls submitCode() interface.

6.6 Submission & Evaluation Module

- Validates:
  - user is logged in
  - contest is active
  - problem belongs to the contest
- Creates new Submission with initial status (e.g., “Pending”).
- Calls evaluateSubmission(submission_id) (async or sync).
- Updates Submission result and forwards to Leaderboard module.

6.7 Judge Engine Module

- For each submission:
  1. Saves code in temporary file.
  2. Compiles code according to selected language.
  3. If compilation fails → result = "Compilation Error".
  4. Else runs against each test case under time limits.
  5. If any test fails → result = "Wrong Answer" or "TLE/RE".
  6. If all pass → result = "Accepted" and full score assigned.
- Sends final result and execution time to backend.

6.8 Leaderboard Module

- Maintains best score for each user and problem.

- Recomputes total score and penalty after each accepted submission.
- Sorts users primarily by total_score, secondarily by penalty_time or earliest success.
- Displays updated ranks to all participants.

Conclusion

This SDD describes all major modules, concurrent processes, data structures, dependencies, interfaces, and detailed design required for the **CODEASTRA 2026 Online Coding Contest Platform**.

It serves as a blueprint for implementation, testing, and future enhancements, ensuring that the system is **modular, scalable, secure, and maintainable**.
